Bags, Vectors & Transformers

Large Language Models & Annotation · Day 4, Part 1

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

Two Ways to Read a Sentence

BERT and GPT use the same building block, attention. The difference is **what each word is allowed to look at** during training.

BERT: fill in the blank

the farmers protested against the **[MASK]** rules

Looks at words on **both sides** of the gap. Built for **understanding**: classifying, comparing, searching.

GPT: what comes next?

the farmers protested against the new **?**

Looks only at the words **so far**, never ahead. Built for **generation**: writing, answering, continuing.

Every position is a training example. One sentence teaches GPT many things at once:

`the` → `farmers` `the farmers` → `protested` `the farmers protested` → `against` ...

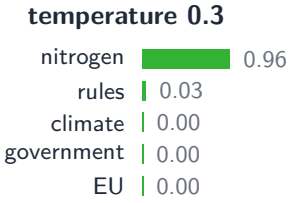
GPT stands for Generative Pre-trained Transformer. "LLM" usually means a very large model of this kind.

Generating Text, One Token at a Time

The loop

1. Read everything so far.
2. Give a probability to **every token** in the vocabulary.
3. Pick one, add it, go back to 1.

It stops at a special **end token**. Long answers mean many trips through the model, so they're slow.



Temperature reshapes the probabilities before the model picks. **Low**: almost always the top token, predictable and repeatable: **what you want for annotation**. **High**: more varied, more mistakes. (Made-up scores for "Dutch dairy farmers protested against the new ...")

From Predictor to Assistant: Three Rounds of Training

1. Pre-training

Predict the next token on **trillions** of tokens: web pages, books, code, papers. Llama 3 read more than 15 trillion. Months on thousands of GPUs.

2. Instruction tuning

Further training on example conversations: a request, and a good answer. Now it **answers** your question instead of continuing your text.

3. Preference tuning

People (or other models) compare two answers; the model is pushed toward the preferred one. This is where its tone and its refusals come from.

On the Hub you'll see both versions. `Qwen2.5-1.5B` is the raw, pre-trained model; it continues text. `Qwen2.5-1.5B-Instruct` went through rounds 2 and 3; it follows instructions. For annotation, you want the instruct version.

Tokens, Size and Cost

GPT models also read **tokens**, pieces of words, like BERT. In English a token is about three quarters of a word; Dutch, names and jargon need more.

Context window

How much fits at once, **prompt and answer together**. GPT-2 (2019): 1,024 tokens. Llama 3.1 (2024): 128,000.

€ Price

Hosted models charge **per million tokens**, in and out. Qwen2.5 72B on OpenRouter in October 2026: $0.36 in, $0.40 out.

Size

1.5 billion parameters: about 3 GB, Colab's free GPU. 8 billion: 16 GB, or 5 GB **quantized** to 4 bits (a laptop). 70 billion: a server or an API.

For annotation: your instructions are sent again with every single text, so a long prompt costs time and money on every item. Ask for a label, not an essay.

What Social Scientists Use LLMs For

Annotation and classification

Topic, stance, sentiment, frames, emotions, incivility. **Today's focus.**

Information extraction

Turning text into data: who said what, which actors, claims, events, dates.

Experimental materials

Drafting vignettes, stimuli and survey items, then pretesting them.

Open answers and interviews

Coding open survey answers; suggesting themes for a human to check.

Translation

Making multilingual corpora comparable, or checking translated survey items.

Simulated respondents

"Silicon samples" (Argyle et al., 2023). Contested: too little variation, unstable over time (Bisbee et al., 2024).

The LLM is a tool inside a research design, never the design itself.

LLMs as Annotators: What the Evidence Says

Cheaper than crowd workers, often as good

ChatGPT, without examples, beat crowd workers on four of five tasks such as stance and frames, at under $0.003 per annotation.

Gilardi, Alizadeh & Kubli (2023), *PNAS*

Across 12 languages

Sentiment, emotions, offensiveness, moral foundations: much better than dictionaries, nearly as good as fine-tuned models.

Rathje et al. (2024), *PNAS*

Not better than a fine-tuned model

On 25 benchmarks, 13 models didn't beat the best fine-tuned classifiers, but agreed fairly well with humans. Advice: LLMs as *members* of annotation teams.

Ziems et al. (2024), *Computational Linguistics*

Wrong conclusions are common

37 annotation tasks redone with 18 LLMs: about **31%** of hypotheses came out wrong with state-of-the-art models, depending on model and prompt.

Baumann et al. (2025), arXiv

Good enough to be useful, not good enough to skip validation.

Ways to Ask: Prompting Strategies for Annotation

	What you add	Trade-off
Zero-shot	Only the task and the list of labels	quickest; the model guesses what you mean
Codebook	A definition per label, plus rules for edge cases	closest to human coding; longer prompt
Few-shot	A handful of coded examples	shows the boundaries; examples steer, for better or worse
Chain-of-thought	"Reason briefly, then give the label"	can help on hard cases; much slower, harder to parse
Label scoring	Read the probability of each label instead of letting it write	always a valid label, plus a confidence; needs the model in Python

What works best depends on task and model: try them on texts you coded yourself.

Anatomy of an Annotation Prompt

```
system:
You code US bill titles by policy area.
Categories:
- Health: health care, insurance, diseases, drugs ...
- Education: schools, teachers, student aid ...
- Taxation: taxes, the tax code, tax credits ...
- Armed Forces and National Security: military ...
If a bill fits several, choose its main purpose.
Reply with the category name only.

user:      Bill title: To repeal the estate tax.
assistant: Taxation

user:      Bill title: {the text to code}
```

Role and task

Codebook: a definition per label

Edge-case rule

Output format

Examples as earlier turns (few-shot)

A The text to code

Standing instructions go in the **system** message; each text arrives as a new **user** message. The same prompt then runs over thousands of texts.

Validate Before You Trust

1. **Hand-code a random sample** yourself, before you look at the model's labels.
2. Split it: a **development** set to try prompts on, a **test** set you use **once**.
3. Compare per label: accuracy, macro-F1, and agreement with human coders.
4. Run it **twice**: does the model agree with itself?
5. Read the errors: model wrong, or codebook unclear?

Why the split matters

Every prompt you try and keep because it scored well is a choice fitted to that data. Try ten prompts on your test set and pick the best, and the score you report is too optimistic. That's one route to the wrong conclusions on the previous slide.

Report: model and version, how you reached it, the full prompt and examples, temperature, date, and the validation scores.

Three Ways to Reach an LLM

Ollama

An app that runs open models **on your own computer**. One download, `ollama pull` a model, and talk to it from Python.

\+ data never leaves your machine; free

– limited by your hardware; open models only

Hugging Face

Download open weights **into Python** with `transformers`, as on Day 3. Runs wherever your code runs: Colab, a cluster, your laptop.

\+ full control: templates, probabilities, settings

– needs a GPU for anything sizable; more code

OpenRouter

One API key and one interface for **hundreds of hosted models**, open and closed, from many providers. You pay per token.

\+ big models, no hardware needed

– your texts go to the provider; models change or disappear

The companies' own APIs (OpenAI, Anthropic, Google, Mistral) work like OpenRouter, for their own models only.

Which Route When?

	Ollama	Hugging Face	OpenRouter
Your texts stay with you	yes	yes, where the code runs*	no
Cost	free	free	per token (often cents)
Models	open, mostly 4-bit	open, full precision	hundreds, open and close
Hardware you need	your laptop	a GPU	none
Rerun it in five years	yes, keep the file	yes, pin the version	not guaranteed
Effort to set up	low	medium	low (an API key)

* In Colab, that's a Google machine. Sensitive data: check your data management plan first.

A sensible workflow: prototype a prompt on a few *non-sensitive* examples with a big hosted model, then run your real data on an open model you control, and validate against human coding either way.

Part 1: The Takeaways

- ✔ GPT predicts the **next token**, one at a time; keep the **temperature** low for measurement
- ✔ Use an **instruct** model; tokens set the context window, the price and the speed
- ✔ Social scientists use LLMs to annotate, extract, translate and draft; **annotation** is the big one
- ✔ Zero-shot, codebook, few-shot, chain-of-thought, label scoring: **test, don't assume**
- ✔ Validate on **your own hand-coded sample**, with a test set you use once
- ✔ Three routes: **Ollama** (local), **Hugging Face** (in Python), **OpenRouter** (hosted)

To the notebook: code bill titles with an LLM in five ways, and see which one agrees best with human coders. **Then Part 2:** using LLMs responsibly, and distillation.